

Project 5: Discover Topics in Text

Grouping 20 Newsgroups documents with TF-IDF and K-Means

1. Executive Summary

This project uses Natural Language Processing (NLP) and K-Means clustering to discover groups of similar documents in the 20 Newsgroups dataset. The text was cleaned and converted into numbers with TF-IDF, then grouped into **K = 5** clusters without using the topic labels. The labels were only used afterwards to check what each cluster contains. PCA was used to visualize the clusters in two dimensions.

Question	Answer
How much data?	11,314 posts loaded; 10,992 kept after cleaning; 20 categories
How was text turned into numbers?	TF-IDF with 5,000 features (99.15% of values are zero)
Value of K	K = 5 (as required); k = 10 tested as an extension
Topics found	Religion, Computers, Sports, Politics and encryption, plus one general cluster
Cleanest clusters	Religion (god, jesus, bible) and Sports (game, team, season)
Main weakness	Cluster 0 holds 49.4% of all posts and mixes many topics
What PCA showed	Computers and religion separate in opposite directions; the rest overlap in the center

Main lesson: K-Means can find real, readable topics in text without any labels, but topic borders are soft. Posts with specific vocabulary (religion, sports, computers) group well, while general or conversational posts end up in one large mixed cluster.

2. Objective and Data

The goal is to answer one question: **can we group documents by topic without telling the algorithm what the topics are?**

Item	Details
Source	fetch_20newsgroups(subset="train") from scikit-learn
Documents loaded	11,314
Documents after cleaning	10,992 (322 became empty and were removed)
Categories	20 newsgroups
Removed parts	Headers, footers and quoted replies

The 20 categories cover computers (5 groups), recreation (autos, motorcycles, baseball, hockey), science (cryptography, electronics, medicine, space), religion (3 groups), politics (guns, Middle East, general) and a for-sale group. Headers, footers and quotes were removed so the model groups posts by their content, not by sender names or group names.

Example documents show the variety: document 0 (rec.autos) asks about a 1970s sports car called a Bricklin, document 5 (talk.politics.guns) debates the definition of weapons in a bill, and document 10 (rec.motorcycles) discusses buying a 1978 Ducati.

Tools: Python 3, pandas, NumPy, scikit-learn, NLTK, matplotlib, seaborn and Google Colab.

3. Methodology

3.1 Text Preprocessing

- Converted all text to lowercase.
- Removed punctuation and numbers.
- Split each post into words (tokenization).
- Removed English stop words (like "the" and "and") and words shorter than 3 letters.
- Removed posts that became empty after cleaning.

3.2 TF-IDF

TfidfVectorizer converted each post into a vector of 5,000 word scores. Only words that appear in at least 5 posts and in no more than 90% of posts were kept.

3.3 K-Means and PCA

- K-Means with K = 5, 10 restarts (n_init = 10) and random_state = 42.
- Top words per cluster from the cluster centroids, plus 3 sample documents per cluster.
- True categories were counted per cluster only to check the results, not to train the model.
- PCA reduced the 5,000 TF-IDF features to 2 components for a scatter plot.

4. Results

4.1 Exploring the Text

The most common words after cleaning were general words such as people (3,997), like (3,885), dont (3,862), just and know. One strange token, maxaxaxax..., appeared 3,317 times; it comes from encoded files pasted into posts and is an example of noise that basic cleaning does not catch.

Why can't we simply give the original text to K-Means?

K-Means groups data by measuring distances between points. Raw text is just words, and there is no way to calculate a distance between two sentences. The text must first be converted into numbers, for example with TF-IDF, so every document becomes a vector that K-Means can compare.

4.2 Converting Text into Numbers

Item	Value
Number of documents	10,992
Number of features	5,000
Share of zero values	99.15%
Sample feature names	cool, cooling, cooper, cooperation, cop, copies, coprocessor, copy, core, correct

A small example with three sentences shows how TF-IDF works: "the" appears in every sentence, so it gets a low weight (0.32), while "fast" appears in only one sentence and gets a higher weight (0.53). On the full data, TF-IDF already highlights topic words per category, for example key, encryption and clipper for sci.crypt, and space, nasa and orbit for sci.space.

4.3 Grouping the Documents (K = 5)

Cluster	Documents	Share of posts
0	5,429	49.4%
1	517	4.7%
2	1,962	17.8%
3	737	6.7%
4	2,347	21.4%

The cluster numbers have no meaning by themselves. K-Means only knows that documents in the same cluster use similar words. Cluster 0 is much larger than the others, which is the first sign that it is a general cluster rather than a single topic.

4.4 Understanding the Clusters

Cluster	Size	Common words	Top real categories (check only)	Possible topic
0	5,429	like, just, new, good, car, use	rec.motorcycles (473), rec.autos (456), sci.electronics (452)	General / catch-all
1	517	god, jesus, bible, christians, believe, christian	soc.religion.christian (319), talk.religion.misc (96), alt.atheism (90)	Religion
2	1,962	thanks, windows, drive, card, know, does	comp.sys.ibm.pc.hardware (395), comp.os.ms-windows.misc (395), comp.sys.mac.hardware	Computers

Cluster	Size	Common words	Top real categories (check only)	Possible topic
3	737	game, team, games, year, players, season	rec.sport.hockey (412), rec.sport.baseball (293), misc.forsale (20)	Sports
4	2,347	people, dont, think, just, know, like	sci.crypt (392), talk.politics.mideast (359), talk.politics.guns (319)	Politics and encryption

Three clusters have clear topic words: religion (god, jesus, bible), computers (windows, drive, card) and sports (game, team, season). Clusters 0 and 4 are built on general words. Cluster 0 (like, just, good) mixes cars, motorcycles and electronics, and cluster 4 (people, dont, think) groups encryption, Middle East and gun posts because they share argument-style language.

Reading sample documents confirms these themes. All three samples from cluster 1 were Christian discussions (prayer, the Bible, a new member describing their faith). Cluster 3 samples were hockey posts, including an NHL box score. Cluster 2 samples discussed monitors and graphics cards. In contrast, the three samples from cluster 0 came from three different groups: space, religion and cars.

Do the documents within each cluster appear to discuss similar topics?

Partly. The smaller clusters are very consistent: 98% of cluster 1 comes from the three religion groups, and 96% of cluster 3 comes from hockey or baseball. Cluster 0 is not consistent, because it holds almost half of all posts from many categories and is built on everyday words rather than topic words.

4.5 Visualizing the Clusters with PCA

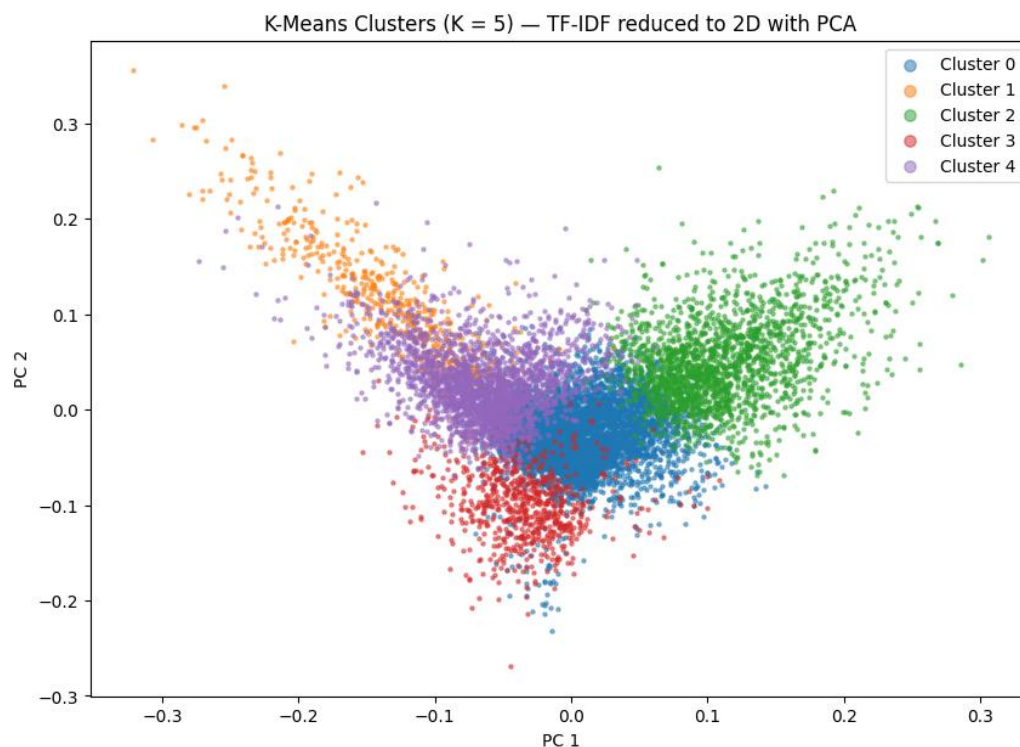


Figure 1. K-Means clusters ($K = 5$) after reducing TF-IDF to 2 dimensions with PCA.

Are some clusters clearly separated?

Yes, partly. Cluster 2 (computers) spreads to the right and cluster 1 (religion) spreads to the upper left, in opposite directions. Cluster 3 (sports) extends downward.

Are some clusters overlapping?

Yes. Clusters 0 and 4 overlap heavily in the center, and all clusters touch near the middle of the plot.

What does this tell us about grouping text documents?

Text topics have soft borders. Documents with specific vocabulary separate well, while general posts share common words and sit in the center. The two components explain only about 1% of the variance, so the plot shows more overlap than really exists in the full 5,000 dimensions.

5. Extension (Beyond the Requirements)

5.1 Choosing the Number of Clusters

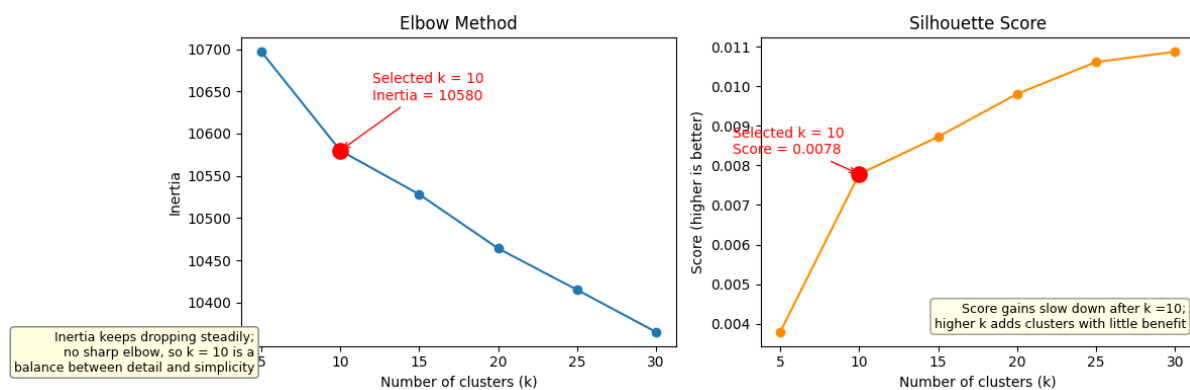


Figure 2. Elbow method and silhouette score for $k = 5$ to 30.

k	5	10	15	20	25	30
Inertia	10,697	10,580	10,528	10,464	10,415	10,365
Silhouette	0.0038	0.0078	0.0087	0.0098	0.0106	0.0109

Inertia drops steadily with no clear elbow, which is common for text data. Silhouette scores are very low for every k , which means the clusters overlap. The silhouette score doubles from $k = 5$ to $k = 10$, so $k = 10$ was tested as an extension.

5.2 K-Means with $k = 10$

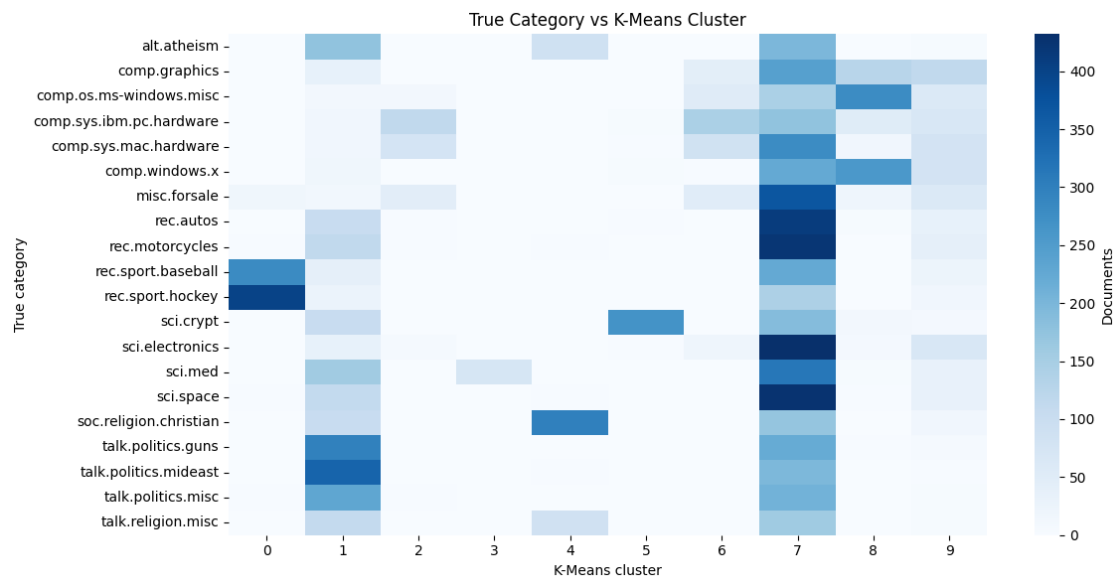


Figure 3. Number of posts from each true category in each $k = 10$ cluster.

Cluster	Size	Top words	Dominant category
0	707	game, team, games, year, players, season, hockey	rec.sport.hockey
1	2,068	people, dont, just, think, like, know, government	talk.politics.mideast
2	268	drive, scsi, drives, hard, disk, floppy, ide	comp.sys.ibm.pc.hardware
3	71	gebcadredslpittedu, chastity, jxp, gordon (email signature)	sci.med
4	483	god, jesus, bible, christians, believe, faith	soc.religion.christian
5	289	key, chip, encryption, clipper, keys, escrow	sci.crypt
6	402	card, video, monitor, bus, cards, mhz, vga	comp.sys.ibm.pc.hardware
7	5,155	like, new, just, good, use, does, car, time	sci.electronics
8	786	windows, file, window, files, program, dos	comp.os.ms-windows.misc
9	763	thanks, email, advance, know, looking, info	comp.graphics

Some topics formed clear clusters: hockey and baseball (cluster 0), religion (cluster 4), encryption (cluster 5) and a small cluster (3) built on one person's repeated email signature in sci.med posts. The politics groups share cluster 1, and the computer groups are split across clusters 2, 6, 8 and 9. Cluster 7 (5,155 posts) is a catch-all that holds most autos, motorcycles, electronics, space and for-sale posts. Against the true labels, the clusters reached $NMI = 0.266$ and $ARI = 0.052$: they carry real topic information but rarely line up one-to-one with the categories.

5.3 Hierarchical Clustering and GloVe

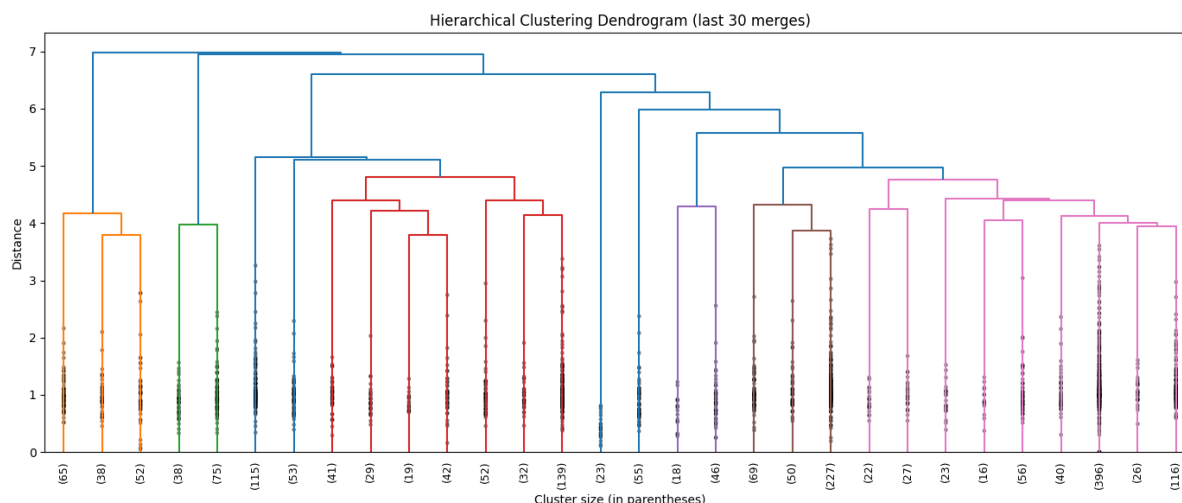


Figure 4. Ward dendrogram of a 2,000-post sample (last 30 merges).

Method	Posts	NMI vs true labels
K-Means + TF-IDF (k = 10)	10,992	0.266
Hierarchical (Ward) + LSA	2,000	0.274
K-Means + GloVe embeddings	10,992	0.326

Hierarchical clustering gave almost the same result as K-Means and found the same themes (sports, religion, hard drives, encryption, Middle East and the email signature). Its purest clusters were sci.med (0.87), sci.crypt (0.84) and Middle East politics (0.81), and it also created one large mixed cluster (722 posts, purity 0.10). K-Means on GloVe word embeddings scored best while using only 50 features instead of 5,000, because embeddings place words with similar meaning close together.

5.4 Predicting Clusters for New Documents

New document	Cluster	Dominant category	Distance	Next closest
The goalie made an amazing save...	0	rec.sport.hockey	0.972	1.002
NASA launched a new satellite...	7	sci.electronics	0.997	1.011
My graphics card driver keeps crashing...	6	comp.sys.ibm.pc.hardware	0.947	0.974
The government should not restrict... firearms	1	talk.politics.mideast	0.999	1.001
Encryption keys and the clipper chip...	5	sci.crypt	0.919	1.002

Three of five new documents went to the matching topic (hockey, graphics card, encryption). The firearms post went to the politics cluster but was almost tied with another cluster (0.999 vs 1.001). The NASA post went to the catch-all cluster, because with $k = 10$ space did not get its own cluster.

5.5 Validation on Unseen Test Documents

Metric	Train	Test (7,296 unseen posts)
Silhouette (higher better)	0.0058	0.0026
Davies-Bouldin (lower better)	8.8505	8.7468
Calinski-Harabasz (higher better)	8.2757	6.7920
Mean cohesion (lower better)	0.9794	0.9825
Mean separation (higher better)	0.9941	0.9939
Mean fit score (higher better)	0.0160	0.0128
NMI vs true labels	0.2657	0.2410
ARI vs true labels	0.0516	0.0447

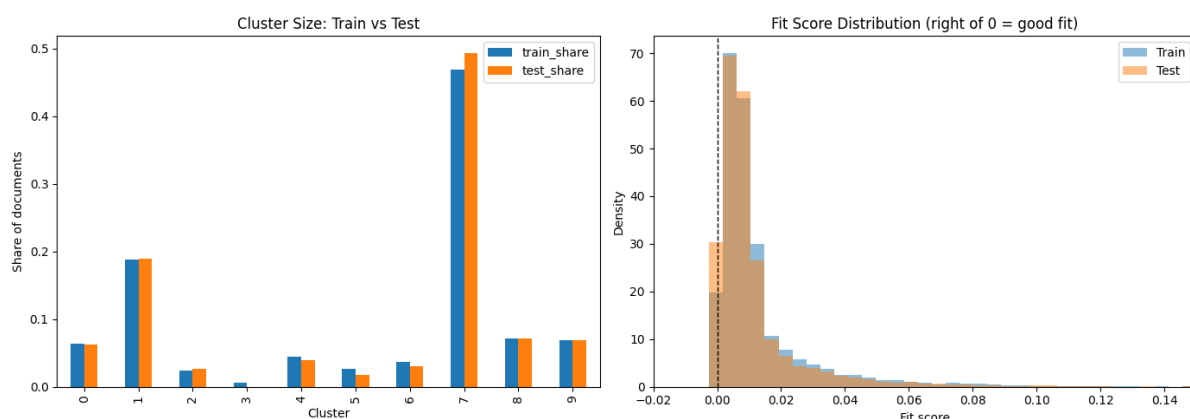


Figure 5. Cluster size on train vs test (left) and fit score distribution (right).

Test results are close to training (NMI 0.266 vs 0.241), so the clusters generalize to posts the model never saw. Cluster shares are also stable; only the catch-all cluster 7 grows slightly (46.9% to 49.4%), and the email-signature cluster 3 gets no test posts, which confirms it was not a real topic. However, cohesion and separation are almost equal, and 96.5% of test posts have a weak fit score (below 0.05), which confirms that cluster borders are soft. The best-fitting posts were one-line messages sitting at the center of the catch-all cluster, so a high fit score does not always mean a clear topic.

6. Final Analysis

1. What is TF-IDF and why is it useful for text data?

TF-IDF (Term Frequency–Inverse Document Frequency) turns each document into a vector of word scores. A word scores high when it appears often in one document but rarely across all documents. This highlights

topic words like "hockey" or "encryption" and lowers the weight of common words, so documents can be compared by their actual content.

2. What value of K did you use?

K = 5, as required. As an extension, k = 5 to 30 was tested with the elbow method and silhouette score, and k = 10 was also run. There was no clear elbow, and silhouette scores stayed very low (0.004 to 0.011) for every k.

3. What topics did you find in the clusters?

Religion (cluster 1: god, jesus, bible), Computers (cluster 2: windows, drive, card), Sports (cluster 3: game, team, season) and Politics and encryption (cluster 4: people, dont, think). Cluster 0 (like, just, good) was a large general cluster, mostly vehicles and electronics posts mixed with short posts from many other groups.

4. Did the documents within each cluster appear similar?

In the smaller clusters, yes. The sports cluster contained hockey scores and baseball discussions, and the religion cluster was mostly Christian and atheism debates. Cluster 0 was less similar because it held about half of all posts (5,429) from many categories.

5. Were some clusters difficult to interpret?

Yes. Cluster 0 was hard to label because its top words (like, just, new, good) are everyday words, and it mixes autos, motorcycles and electronics. Cluster 4 combined encryption, Middle East and gun politics, which are different subjects that share debate-style vocabulary like "people", "think" and "dont".

6. What did the PCA visualization show?

Computers and religion spread in opposite directions and were the most separated. Sports extended in its own direction, while the general cluster and the politics cluster overlapped heavily in the center. The two components explained only about 1% of the variance, so the plot shows more overlap than really exists.

7. What is one limitation of using K-Means for text?

K-Means puts each document in exactly one cluster, but many posts mix topics, for example encryption and government policy. It also needs K chosen in advance, and in high-dimensional sparse TF-IDF data the distances between documents are very similar, so clusters overlap (silhouette 0.0038 at k = 5).

7. Limitations and Next Steps

- With K = 5, related newsgroups are forced together, so a perfect match with the 20 categories is not possible.
- Noise survived cleaning, such as encoded text (maxaxax...) and a repeated email signature that formed its own small cluster.
- Very short posts carry almost no information but still get a cluster.
- Next steps: remove signatures and very short posts, add lemmatization (drive, drives, driving), cluster on LSA-reduced TF-IDF, and try topic models such as LDA or NMF that allow a post to belong to more than one topic.

8. Conclusion

This project converted 10,992 newsgroup posts into TF-IDF features and grouped them with K-Means (K = 5) without using any labels. Four clusters had clear themes (religion, computers, sports, and politics and encryption), and one large cluster collected general posts. PCA showed that some topics separate well while most documents overlap near the center. The key takeaway is that text can be turned into numbers and grouped by machine learning: K-Means does not know the topics in advance, but it discovers useful groups based on the words documents share.

Appendix: Reproducibility

- Notebook: Project_5_Basic_NLP_with_Unsupervised_Learning.ipynb (GitHub link in the page header).
- Data: 20 Newsgroups via scikit-learn fetch_20newsgroups, with headers, footers and quotes removed.
- Settings: TF-IDF max_features = 5,000, min_df = 5, max_df = 0.9; K-Means n_init = 10; random_state = 42.
- To reproduce: open the notebook in Google Colab and run Runtime → Restart session and run all.